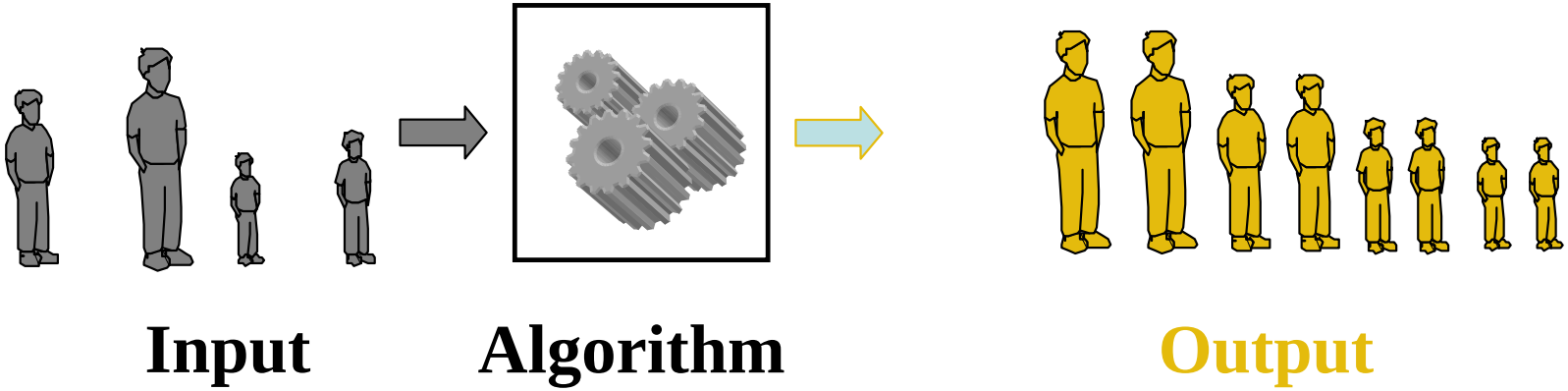
Ziele dieser Semesterwoche 1

Sie können:

- Die Komplexität einer Codesequenz analysieren und mit der O-Notation klassifizieren und beschreiben.
- Ein rekursives Problem mit Java resp. Python lösen.

Analyse von Algorithmen



Ein **Algorithmus** ist eine *step-by-step* Anweisung zum Lösen eines Problemes mit begrenztem Zeitaufwand.

Relevante *primitive Operationen* für die Analyse:

- Evaluation eines Ausdrucks
- Zuweisung eines Wertes an eine Variable
- Indexieren eines Arrays
- Aufruf einer Methode
- Verlassen einer Methode

Beispiel: Finde grösstes Element in einem Array

- Beschreibung des Algorithmus in *Pseudo-Code*
- Zählen der *primitiven Operationen* für den *Worst-Case*

Algorithm arrayMax(A, n)	# operations
currentMax $\leftarrow$ **A**[0]	2
for i $\leftarrow$ 1 **to n** – 1 **do**	1 + 2**n**
if A[**i**] > **currentMax then**	2(**n** – 1)
currentMax $\leftarrow$ **A**[**i**]	2(**n** – 1)
{ increment counter i }	2(**n** – 1)
return currentMax	1
Total (Worst-Case)	8**n** – 2

Worst Case:

2 + (1 + 2n) + 3 * 2(n-1) + 1 ⇨ 8n - 2 primitive Operationen

Best Case:

2 + (1 + 2n) + 2 * 2(n-1) + 1 ⇨ 6n primitive Operationen

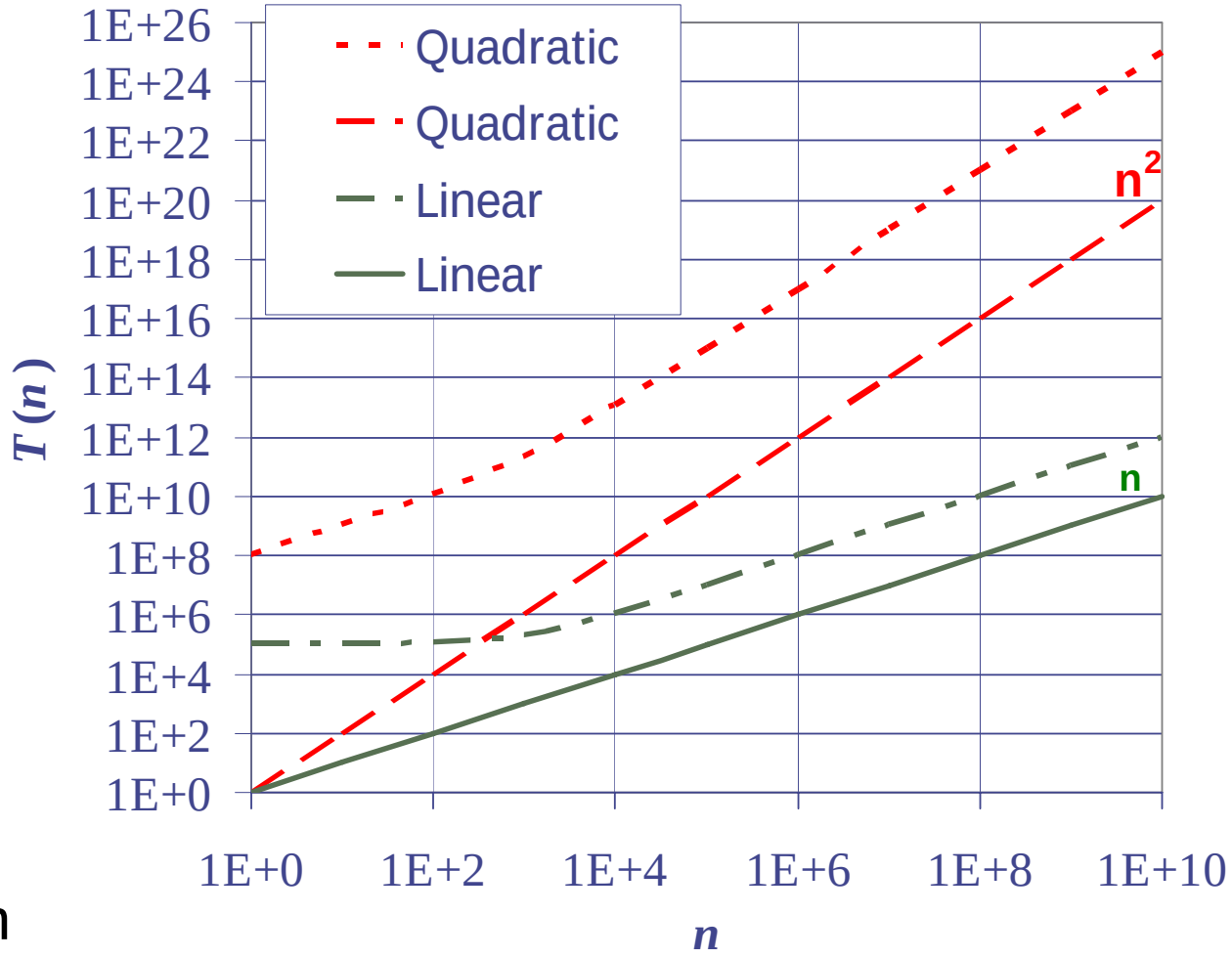
Konstante Faktoren

Das Laufzeitverhalten für grosse ***n*** ist nicht beeinflusst von

- Konstanten Faktoren
- Tieferen Potenzen (höchste Potenz überwiegt)

Beispiel:

- $10^2 n + 10^5$ ist eine lineare Funktion
- $10^5 n^2 + 10^8 n$ ist eine quadratische Funktion



O-Notation

Mit der *O-Notation (Big-O-Notation)* wird eine qualitative Aussage über das Laufzeit-Verhalten eines Algorithmus ab einer bestimmten Anzahl Elemente gemacht.

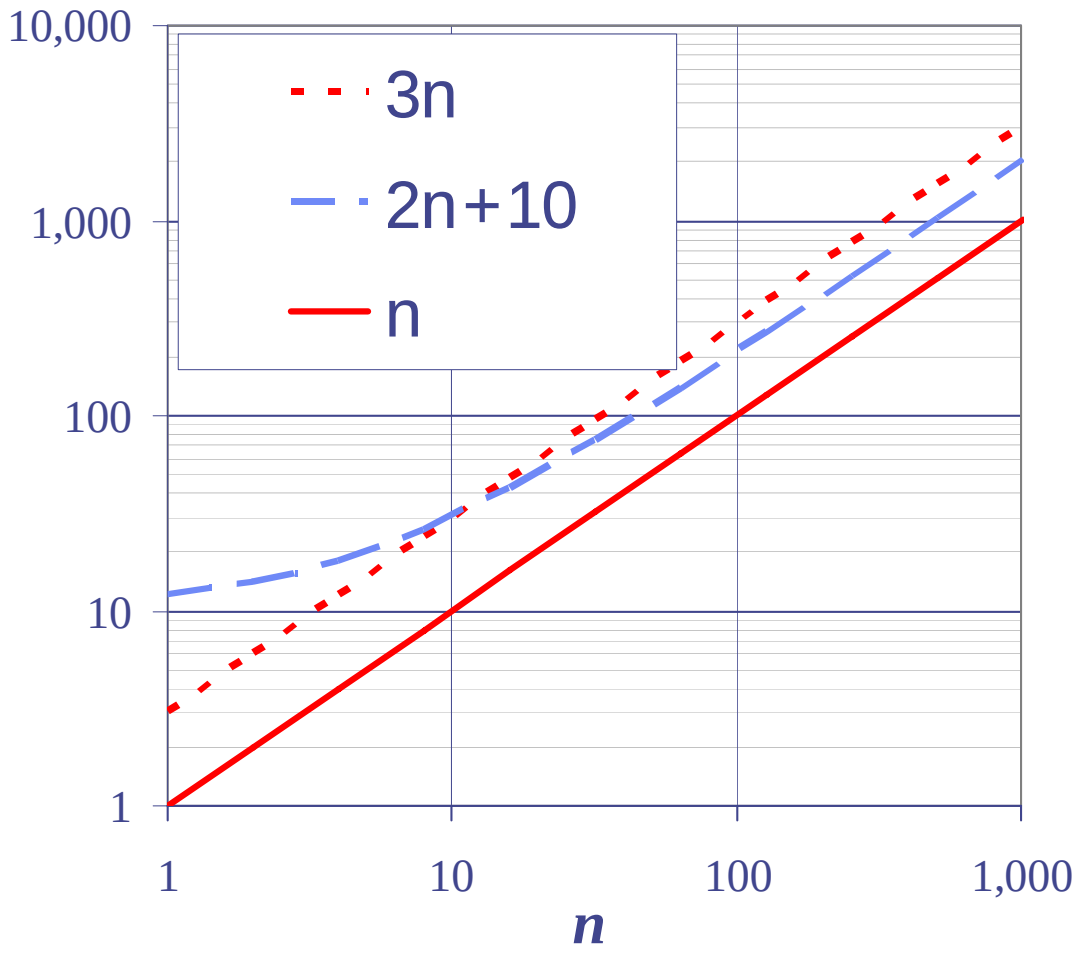
Gegeben:
Funktion ***f***(***n***) und ***g(n)***

Wir sagen:
f(***n***) ist ***O(g(n))***
wenn es eine positive Konstante ***c***
und ein $\boldsymbol{n_0}$ gibt, so dass:

$\boldsymbol{f(n) \leq c * g(n)}$ für $\boldsymbol{n \geq n_0}$

Somit:
$2n + 10 \leq 3 * n$, *ab* $n \geq 10$
2n + 10 ist ***O(n)*** für $n_0=10$

($f(n) = 2n + 10;\ c = 3;\ g(n) = n$)

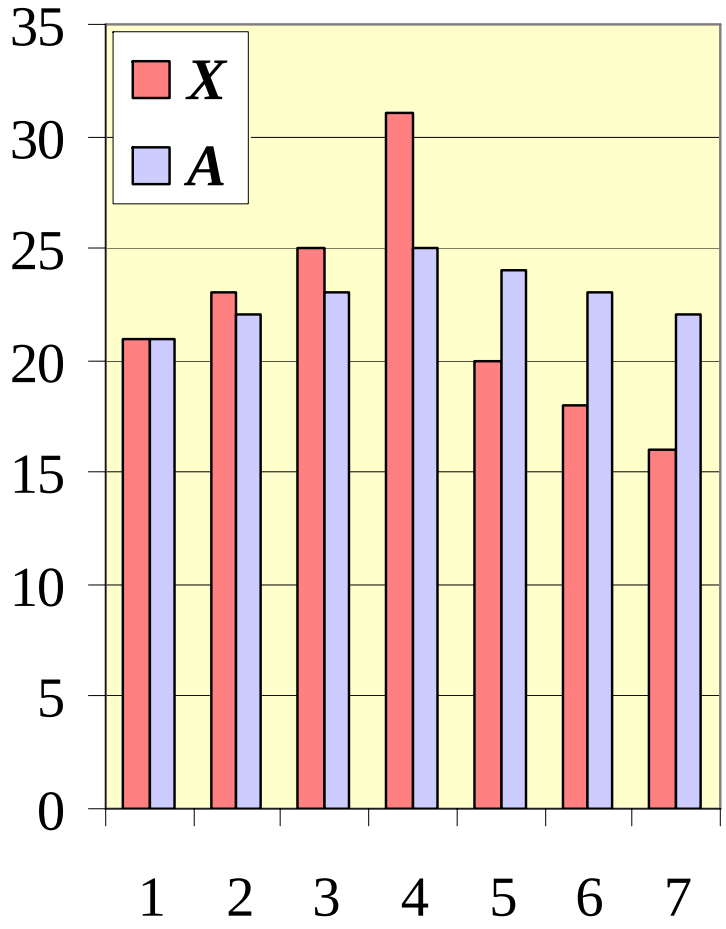


Präfix-Durchschnittsberechnung

Der n-te *Präfix-Durchschnitt* im *Array X* ist der Durschnitt der ersten *n* Elemente des Arrays X.

Die *Präfix-Durchschnitte* werden im *Array A* gespeichert:

A[i] = (**X**[0] + **X**[1] + … + **X**[i]) / (i+1)



Hinweis:
Präfix-Durchschnittsberechnung wird z.B. bei Finanz-Analysen eingesetzt.

Der folgende Algorithmus berechnet den Präfix-Durschnitt mit quadratischerLaufzeit:

```
Algorithm prefixAverages1(X, n)
    Input array X of n integers
    Output array A of prefix averages of X        #operations
    A ← new array of n integers                   n
    for i ← 0 to n – 1 do                         n
        s ← X[0]                                  n
        for j ← 1 to i do                         1 + 2 + ...+ (n – 1)
            s ← s + X[j]                          1 + 2 + ...+ (n – 1)
        A[i] ← s / (i + 1)                        n
    return A                                      1
```

Beachte:

Es werden aufgrund der Erkenntnis, dass Konstanten auf die O-Abschätzung keinen Einfluss haben, keine Konstanten mehr dargestellt, z.B. bei der ***for***-Schleife.

Arithmetische Progression

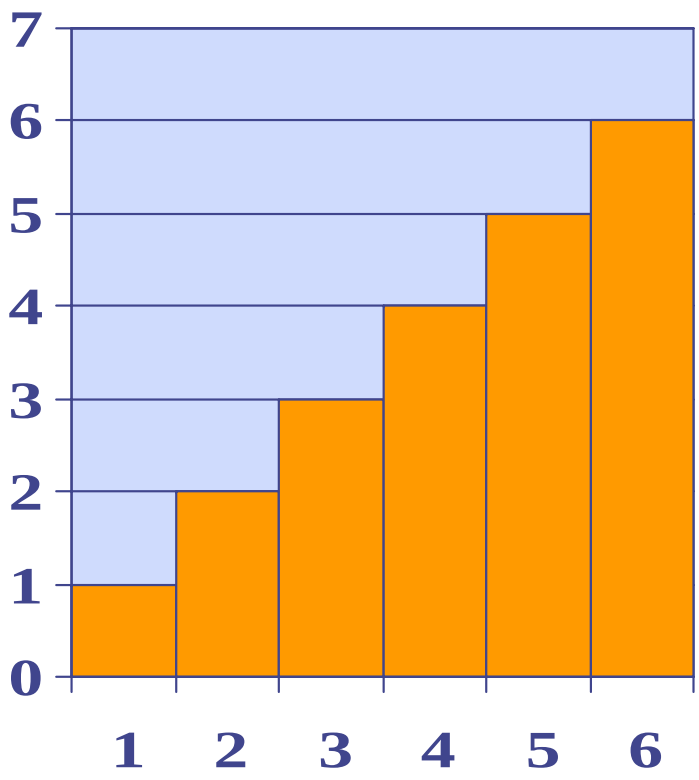
Die Laufzeit von *prefixAverages1* ist:

O(1 + 2 + 3 + ... + n) ⇨ $\sum_{i=1}^{n} i$

Die Summe von ***n*** ganzen Zahlen ist:

$$\frac{n^2}{2} + n\frac{1}{2} = \frac{n^2+n}{2} = \frac{n(n+1)}{2}$$

⇨ *Gausssche Summenformel*
(siehe Skizze rechts)



Somit:
prefixAverages1 hat eine Laufzeit von: **O(n²)**

Hinweis:
Das ist eine typische Situation bei verschachtelten Schleifen!

Selection-Sort

Selection-Sort (Sortierung durch direktes Auswählen) ist ein elementares Sortierverfahren.

Dabei werden die zu sortierenden Daten zuerst in einer unsortierten Sequenz gespeichert.

Dann wird jeweils gezielt das kleinste Element aus der Sequenz gesucht.

Laufzeitverhalten von *Selection-Sort* :

Einfügen von *n* Elementen mit *n insert*-Operationen ⇨ **O(n)**

Entfernen von *n* Elementen in dem die ganze, übrig gebliebene unsortierte Sequenz nach dem kleinsten Element durchsucht wird:

$$n + n-1 + n-2 + \ldots + 2 + 1 = \sum_{i=1}^{n} i \Rightarrow \mathbf{n * (n+1) / 2} \Rightarrow \mathbf{O(n^2)}$$

Somit: Laufzeit von *Selection-Sort* ⇨ **$O(n^2)$**

Beispiel:

		Unsorted Sequence
Input:	(7,4,8,2,5,3,9)	()
Phase 1: Insert		
(a)	(4,8,2,5,3,9)	(7)
(b)	(8,2,5,3,9)	(7,4)
..	.. ..	
. .	.	
(g)	()	(7,4,8,2,5,3,9)
Phase 2: Remove		
(a)	(2)	(7,4,8,5,3,9)
(b)	(2,3)	(7,4,8,5,9)
(c)	(2,3,4)	(7,8,5,9)
(d)	(2,3,4,5)	(7,8,9)
(e)	(2,3,4,5,7)	(8,9)
(f)	(2,3,4,5,7,8)	(9)
(g)	(2,3,4,5,7,8,9)	()

Insertion-Sort

Insertion-Sort (*Sortierung durch direktes Einfügen*) ist wie *Selection-Sort* ein elementares Sortierverfahren.

Dabei werden die zu sortierenden Daten zuerst in eine sortierten Sequenz eingefügt.
Dann wird jeweils das erste Element aus der Sequenz entnommen.

Laufzeitverhalten von *Insertion-Sort* :
Einfügen von *n* Elementen mit *n insert*-Operationen indem bei jedem *insert* das neue Element an der richtigen Position in der sortierten Sequenz eingefügt wird.

$$1 + 2 + \ldots + n-2 + n-1 + n = \sum_{i=1}^{n} i \Rightarrow \mathbf{n * (n+1) / 2} \Rightarrow \mathbf{O(n^2)}$$

Entfernen von *n* Elementen indem jeweils das erste Element aus der Sequenz entnommen wird: ⇨ **O(n)**

Somit: Laufzeit von *Insertion-Sort* ⇨ $\mathbf{O(n^2)}$

Beispiel:

		Sorted Sequence
Input:	(7,4,8,2,5,3,9)	()
Phase 1: Insert		
(a)	(4,8,2,5,3,9)	(7)
(b)	(8,2,5,3,9)	(4,7)
(c)	(2,5,3,9)	(4,7,8)
(d)	(5,3,9)	(2,4,7,8)
(e)	(3,9)	(2,4,5,7,8)
(f)	(9)	(2,3,4,5,7,8)
(g)	()	(2,3,4,5,7,8,9)
Phase 2: Remove		
(a)	(2)	(3,4,5,7,8,9)
(b)	(2,3)	(4,5,7,8,9)
..	..	..
.	.	.
(g)	(2,3,4,5,7,8,9)	()

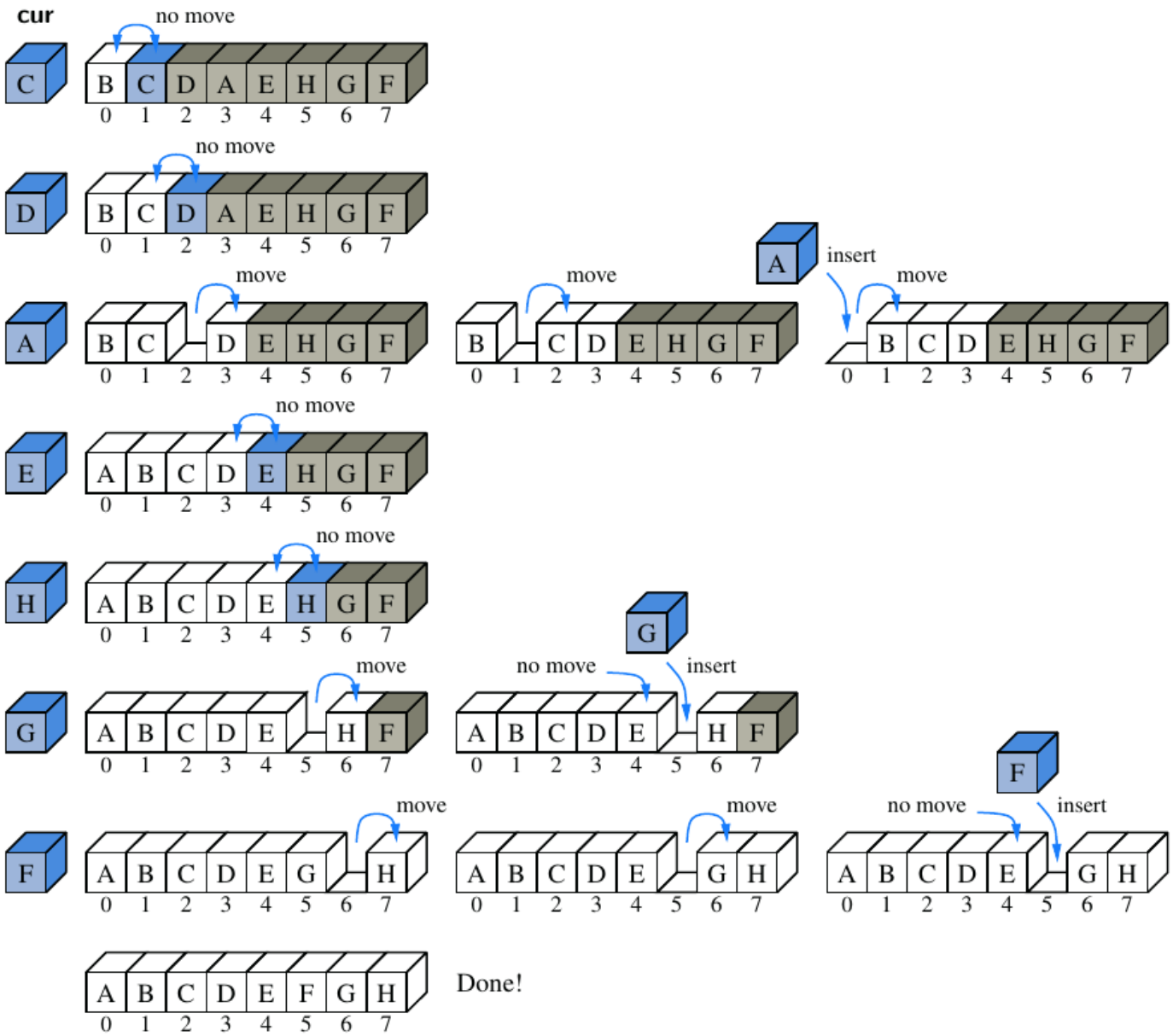
In-Place Insertion-Sort (1/2)

Anstelle einer zusätzlichen, internen Datenstruktur (*Sequence/Liste*), können wir den *Insertion-Sort* auch sog. *In-Place* realisieren, d.h. direkt in der übergebenen Datenstruktur:

- der linke Teil des Arrays entspricht der *‚Sorted Sequence'*
- der rechte Teil des Arrays enthält die noch nicht richtig einsortieren Elemente
- wir nehmen jeweils das nächste Element aus der rechten Seite (unsortiert) und speichern es (⇨ *cur*)
- mit *move* -Operationen schaffen wir Platz für die richtige Position dieses nächsten Elements (*cur*) im linken Teil

Zu Beginn hat der linke, sortierte Teil des Arrays bereits die Länge 1, d.h. das erste Element wird bereits als *'sortiert'* betrachtet.

In-Place Insertion-Sort (2/2)



Rekursion

Iteration : Ein Algorithmus enthält Abschnitte, die mehrfach durchlaufen werden.

Rekursion : Ein Algorithmus enthält Abschnitte, die sich selbst direkt oder indirekt aufrufen.

Iteration ⇨ Schleife (*for, while, do while*)

Direkte Rekursion ⇨ Methode ruft sich selbst wieder auf

Indirekte Rekursion ⇨ Methode *A* ruft Methode *B* auf. Innerhalb dieses Aufrufes wird wieder die Methode *A* aufgerufen.

Klassisches Beispiel: *Fakultät* n! = 1· 2· 3· ··· · (n-1)· n

Rekursive Definition:

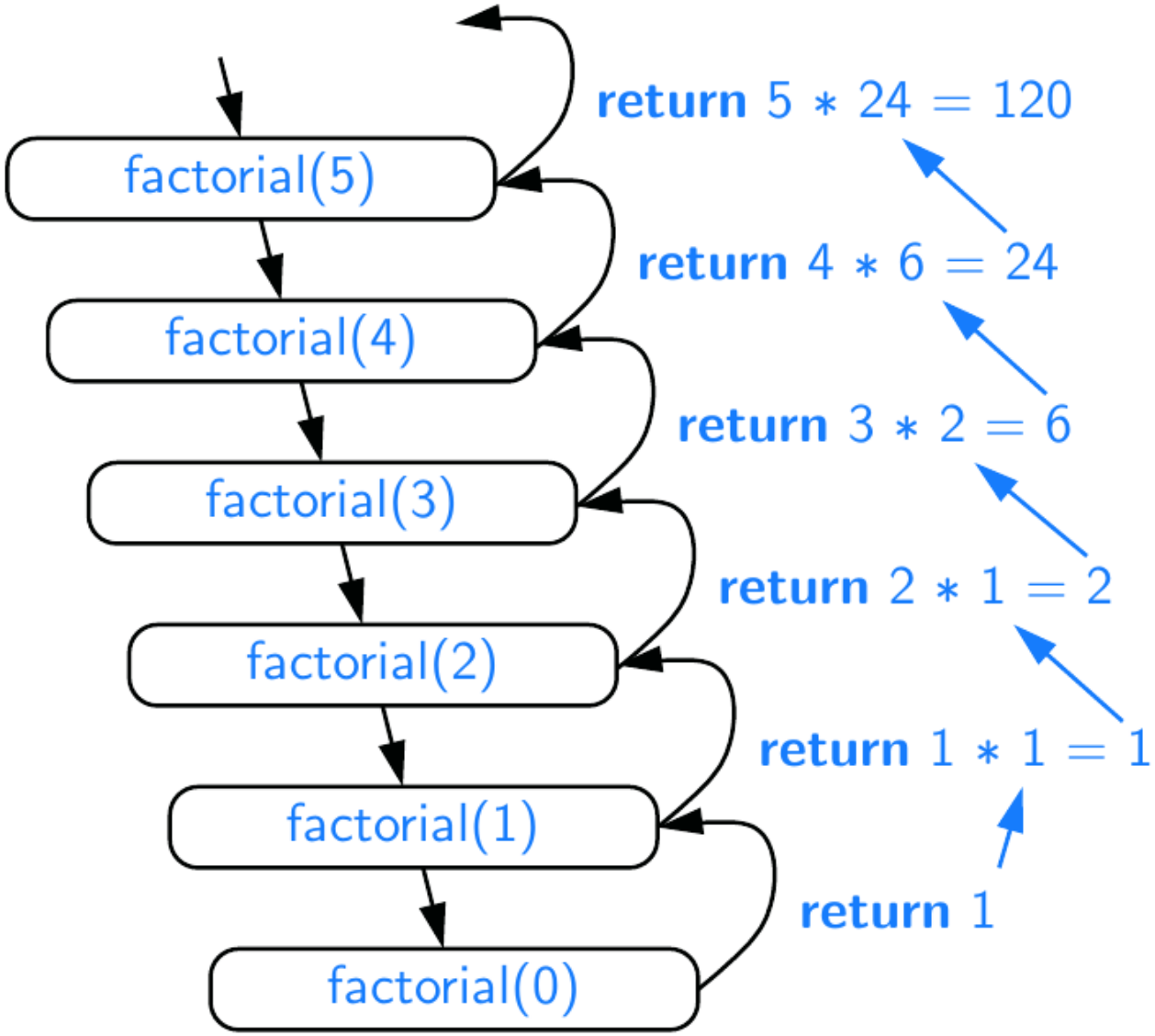
$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot f(n-1) & else \end{cases}$$

Wichtig: ***Verankerung*** (*Endbedingung*, *base case*)

Als Java-Methode:

```
static int factorial(int n) {
  if (n == 0)
    return 1;
  else
    return n * factorial(n-1);
}
```

Als Python-Funktion:

```
def factorial(n):
  if n == 0:
    return 1
  else:
    return n * factorial(n-1)
```

Beispiel: *5! : 1 * 2 * 3 * 4 * 5 = 120*